Understanding Permission Usage and Authorization in MCP Tools for LLM Agents

Anonymous Authors

***Abstract*—Modern Large Language Model (LLM) agents are increasingly connected to a growing ecosystem of external tools. The Model Context Protocol (MCP) standardizes this connection, allowing agents to invoke diverse external capabilities through a common tool layer. Yet this convenience also places security-sensitive permissions within agent reach, creating risks that remain insufficiently understood. This paper presents a large-scale empirical study of permission usage and authorization mechanisms in the MCP ecosystem. The study pipeline combines static analysis with LLM-assisted semantic analysis to identify concrete external-API invocations in each MCP tool. Using these invocation behaviors as evidence anchors, we then analyze critical permission usages, authorization mechanisms risks, and mismatches between tool descriptions and implementation. Applying this pipeline, our study identifies 27,465 permission-relevant external APIs across 35,476 MCP tools from 4,049 MCP servers. The results reveal that many MCP tools access high-impact capabilities without enforcing fine-grained authorization checks. Moreover, 2,219 tools' descriptions fail to faithfully disclose the permission behaviors actually exercised during tool executions, causing two LLMs to misjudge their required permissions. Together, these findings reveal an underexamined MCP privilege surface, where high-impact permissions remain insufficiently governed and disclosed.**

I. Introduction

Agents of large language models (LLMs) are increasingly connected to real-world software resources through external tools, and the Model Context Protocol (MCP) is rapidly becoming a standard layer for exposing these capabilities to agents. To complete user tasks, agents may retrieve data, operate databases, manage cloud resources, or execute local commands through tool calls [1], [2]. MCP standardizes this interaction by providing a common interface for tool discovery and invocation [3]. As a result, many open-source MCP servers have been developed and released across data access, software development, and infrastructure management scenarios [4], [5]. This growth makes the permission control of MCP tools a security-critical part of agent systems.

Problem. When an agent invokes an MCP tool, it sees the tool interface but not the server-side code that will run. The agent usually selects a tool based on its name, input schema, and natural-language description. These fields describe *what the tool is supposed to do*, but they do not necessarily reveal *what operations the implementation actually performs* through external API calls. For example, as shown in Section IV-D, a benign tool described as a safe directory-listing command can still make network requests, write files, and collect host information during execution. Such a mismatch does not necessarily mean that the tool is malicious. Instead, tool descriptions often summarize the user-facing task while omitting permission-relevant implementation steps. As a result, agents or users may approve a tool call without knowing the permissions that will actually be exercised.

Research Gap. Despite growing concern about tool-using agents, we still lack a systematic understanding of how MCP tools exercise and expose permissions in their actual implementations. Existing studies have characterized agent architectures, tool-use workflows, MCP-style protocols, and broad safety risks such as harmful tool execution, prompt injection, and data leakage [6]–[12]. These studies show that external tool use can be risky, but they mainly reason at the level of agent behavior or attack scenarios [13]–[16]. However, there is still no systematic study of which external APIs each MCP tool uses, what authorization protects those APIs, and whether agents can infer these permission behaviors from the tool description before invocation.

Challenges. To study MCP permissions at scale, we must infer the permissions exercised by each MCP tool from server implementations rather than from metadata exposed through the MCP interface. We observed that permission-relevant operations in MCP servers are typically implemented through external API or SDK calls. Thus, large-scale MCP permission analysis must both identify API calls associated with each tool *within heterogeneous server implementations* and interpret those calls with semantic information *beyond the server implementation itself*. This requirement poses two key challenges. First, MCP servers do not explicitly declare which external API calls are reachable from each exposed tool. These servers differ in programming languages, SDK versions, tool registration patterns, and dispatch logic, making it difficult to identify API calls associated with each tool across heterogeneous implementations. Second, external API calls in MCP server code provide only incomplete evidence for permission analysis. The code usually exposes the call expression, imported package, and local arguments, while the permission effect and required authorization are often defined by downstream service behavior and external documentation.

Empirical Study. This paper presents a large-scale empirical study of permission usage and authorization in the MCP ecosystem. To support this study, we develop a multi-stage auditing pipeline. The pipeline first collects and normalizes open-source MCP servers from MCP directory websites (e.g., MCP.so [17] and Cursor Directory [18]). To address the challenge of heterogeneous implementations, the pipeline builds a unified CodeQL-based abstraction over MCP server code. This abstraction identifies exposed tools, constructs tool-rooted

call graphs, and locates external API or SDK calls associated with each tool. To address the limited permission evidence available in the server code, it performs permission analysis over the recovered tools and API calls. This analysis combines LLM-assisted semantic analysis with documentation retrieval to infer permission-relevant behavior. It further characterizes authorization mechanisms with code-agent analysis and assesses description-permission consistency.

We analyze 4,049 open-source MCP servers, extract 35,476 tools, and identify 27,465 permission-relevant external API calls. Based on these implementation-derived API invocations, the study centers on the following four research questions.

- RQ1. In what deployment scenarios are MCP servers used, and how complex are their code and tools?
- RQ2. What permission-relevant APIs do MCP tools invoke, and which operations pose elevated security risks?
- RQ3. How do MCP servers authorize the external permissions exercised by their tools, and how closely do these practices align with the least-privilege principle?
- RQ4. How accurately do MCP tool descriptions reflect underlying permission behavior, and can LLMs infer permission scopes from these descriptions?

Our results show that current MCP deployments expose agents to a broad and often under-governed permission surface. First, MCP servers are already used in security-sensitive scenarios, and 123 servers invoke more than 100 permission-relevant APIs, concentrating multiple capabilities behind a single agent-facing interface. Second, high-impact API categories account for 63.1% of all identified permission-relevant APIs, posing a mismatch between APIs designed for controlled application workflows and their new role as dynamically selected agent-callable tools. Third, authorization remains largely absent or coarse-grained, with 3,555 servers (87.8%) relying on static tokens, passwords, or no authorization. Fourth, 2,219 tools (6.3%) of MCP tools have descriptions that fail to expose permission-relevant behavior, creating blind spots during agent tool calls and increasing the risk of permission misuse.

Together, these findings reveal two systemic gaps in current MCP deployments. At the development level, MCP servers often concentrate permission-relevant operations and expose them through incomplete descriptions, making the tool authority difficult for agents and users to inspect before invocation. At the ecosystem level, existing API and authorization designs remain oriented toward human users and fixed applications, rather than autonomous agents that dynamically select and combine tools. Improving MCP security therefore requires both clearer permission-aware development guidelines for MCP server developers and broader ecosystem support for fine-grained, agent-aware API authorization.

In summary, this paper makes the following contributions:

- We develop a multi-stage auditing pipeline that uses API invocations as evidence anchors to identify permission-relevant behavior, authorization patterns, and description-permission mismatches in MCP tools.

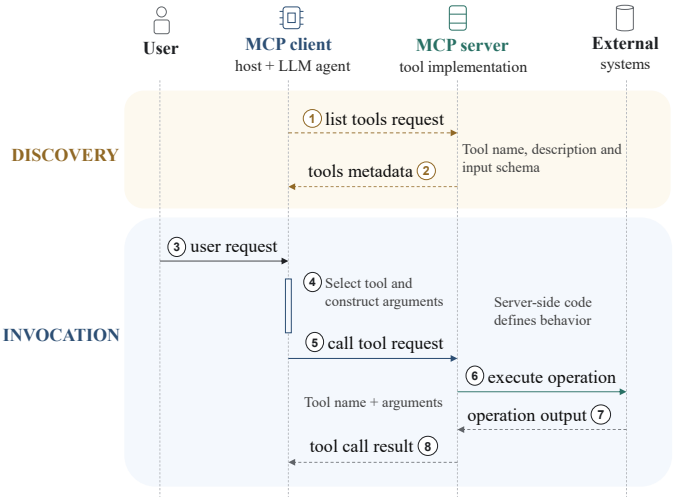


Fig. 1: Overview of MCP Interactions.

- We conduct a large-scale empirical study of the MCP permission layer, analyzing 4,049 open-source servers, 35,476 tools, and 27,465 permission-relevant APIs. Our code and data for reproducing this study are provided at: https://anonymous.4open.science/r/MCP-Server-Understanding-4509/.
- We characterize permission risks in current MCP practice, showing that high-impact tools are often paired with coarse-grained authorization and incomplete descriptions.

II. Background

A. MCP Architecture and Tool Calls

MCP provides a standardized interface for connecting LLM agents to external tools and services [3]. In an MCP-based application, a host or MCP client connects to one or more MCP servers and discovers the tools exposed by each server. As shown in Figure 1, this discovery stage begins with a list-tools request and returns tool metadata. Each tool is described by a name, an input schema, and a natural-language description [3]. This metadata forms the agent-visible interface, telling the model which tools are available, how to invoke them, and what each tool appears to do.

After discovery, the model selects a tool from the advertised metadata and constructs a tool-call request. The MCP client sends this request to the corresponding MCP server, where server-side code executes the selected tool and returns the result. During execution, the tool implementation may call external interfaces such as system APIs, service APIs, or resource clients. These calls access external systems and are where permission-relevant behavior is exercised. [19]–[21]. Thus, the MCP workflow separates what the agent can observe from what the tool actually does.

B. Permission Control and Authorization

For tool-integrated LLM agents, permission control determines what external actions a model-selected tool can perform. The Principle of Least Privilege (PoLP) [22] requires each principal or component to receive only the permissions needed

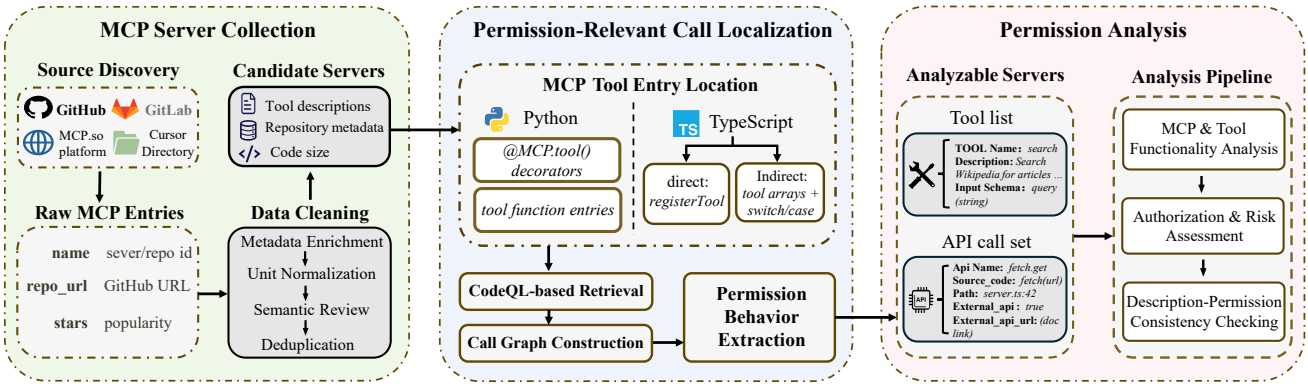


Fig. 2: Workflow of the Empirical Study.

for its intended function, limiting the impact of failures, misuse, or compromise [23], [24]. In this paper, permission denotes the operation-level capability that an MCP tool implementation can exercise through external API calls, rather than what its name, description, or declared interface suggests.

Authorization mechanisms enforce permission boundaries, but their granularity varies. Traditional approaches often rely on credentials such as usernames and passwords, API keys, or static tokens. These credentials are simple to deploy, but they typically bind access to an account or broad API surface. OAuth 2.0 provides a more explicit delegated authorization model, allowing a client to obtain a temporary and scoped access token [25], [26]. Compared with static credentials, OAuth can support limited lifetimes, delegated access, and scope annotations [27].

MCP introduces OAuth-based authorization for protected servers, where a client obtains an access token and uses it to invoke protected tools. This mechanism standardizes authorization between the agent-side client and the MCP server. However, it does not directly govern how server-side tool implementations access downstream external systems during execution [28], [29]. This study therefore focuses on the permissions exercised by MCP tools through their implementations and external API calls.

III. Study Pipeline

Figure 2 shows the three-stage workflow of our empirical study. The pipeline uses concrete API invocations as implementation evidence for analyzing permission usage and authorization. Specifically, we first collect and normalize open-source MCP servers from MCP directory websites and GitHub repository lists. Second, the pipeline builds a unified CodeQL-based abstraction over heterogeneous MCP implementations to recover exposed tools and the external API calls associated with them [30], [31]. Third, it conducts permission analysis over the recovered tools and API calls.

A. MCP Server Collection

The pipeline first collects MCP servers from diverse sources. Specifically, it collects publicly accessible MCP server entries from MCP.so, the Cursor Directory MCP index, and GitHub-based aggregation repositories. These sources cover both dedicated MCP directories and community-maintained repository lists, allowing us to capture servers that are published through different ecosystem channels.

During collection, we addressed two common sources of ambiguity. First, the repository-level view does not always align with the server-level unit of analysis. A single repository may contain multiple independent MCP servers, while another may include an MCP server only as a submodule within a larger project. We resolved these cases through LLM-assisted semantic inspection with manual review. Second, some collected entries did not correspond to executable MCP server implementations. We therefore removed entries that only provided tutorials, installation scripts, configuration examples, or other MCP-related materials. After normalization, filtering, and deduplication, we obtained 4,049 MCP servers.

B. Permission-Relevant Call Localization

This step locates external API calls reachable from MCP tool implementations. It first identifies tool entry points, then constructs call graphs from these entries and extracts all reachable function calls. The pipeline then performs semantic analysis with online retrieved context to infer concrete permission behavior and exclude calls unrelated to permissions. The resulting permission-relevant APIs form the evidence base for later permission analysis.

To construct these call sites, we first need to identify the entry point of each tool. We recover these tool entry points with CodeQL analyses for Python and TypeScript projects. For Python projects, MCP tools are commonly defined through decorators such as `@MCP.tool()`. However, for TypeScript projects, tool exposure is not always tied to a single implementation function. In the direct case, tools are registered through `registerTool`. In the indirect case, a customized router is implemented to route the request according to the requested tool name. To recover these indirect entry points, we analyze tool arrays and switch-case branches that implement this name-based dispatch.

Next, starting from each tool entry point, we connect function calls to build a tool-rooted call graph. This step

uses CodeQL to construct a unified call-graph representation across supported languages. From this graph, the pipeline collects external-library API calls that may correspond to permission behavior involving the local machine, databases, cloud services, or other downstream resources.

The recovered calls are candidates rather than final permission-relevant APIs. The pipeline therefore extracts the permission behavior associated with each candidate API invocation. This step uses LLM-based semantic analysis, but model hallucination may occur when analyzing rarely used APIs or APIs with similar names. To mitigate this problem, we observe that many external APIs are accompanied by publicly available documentation. We therefore augment the LLM analysis with online documentation retrieval, using the API name and package context as search cues. This allows us to clarify the API's functionality, reveal the permission behavior it may exercise, and exclude calls that do not involve permission-bearing operations, such as `string.join()`. Finally, the pipeline produces a set of permission-relevant API calls together with corresponding behavior descriptions.

C. Permission Analysis

The third stage takes as input the recovered tools and corresponding permission-relevant API calls, and uses them as evidence to support the downstream empirical analysis. This stage consists of three steps: MCP and tool functionality analysis, authorization and risk assessment, and description-permission consistency checking.

For functionality and permission-category analysis, we use tool lists, descriptions, and invoked APIs to infer and classify server scenarios and permission-relevant API categories. To construct the classification categories, we use repeated sampling to elicit candidate categories and then merge semantically overlapping ones. Specifically, we repeatedly sample 50 instances and provide their corresponding normalized context for semantic grouping. We repeat this procedure 100 times to reduce sampling variance, aggregate the labels proposed across runs, and merge synonymous or overlapping labels. This procedure retains eight semantically distinct categories for MCP servers and ten categories for permission-relevant APIs. A manual review confirmed that these categories provide broad coverage while remaining mutually distinct.

Second, we analyze the authorization mechanisms used by MCP tools when they invoke permission-relevant external APIs. Because authorization flows may be implemented through highly diverse code patterns, we use a code agent, i.e., Copilot, to identify the authorization type. Specifically, we provide Copilot with the project `README.md`, tool names, and invoked APIs, and ask it to connect authorization-relevant APIs to the corresponding flows, thereby determining the authorization type. We sample 20% of the MCP servers and manually review the results to ensure that the code agent correctly identifies the authorization type. After manual review, we find that all the sampled servers are correctly classified.

Lastly, we evaluate description-permission consistency for each tool by comparing its actual permission behavior with the permission categories inferable from its visible name, description, and input schema. Since an LLM agent can observe only this metadata before invocation, this step estimates what permission scope is transparent at tool-selection time. Specifically, we provide LLMs with only the information available through the MCP interface and ask whether the tool's invoked APIs would be expected when using that tool. This analysis uses two widely used tool-calling LLMs, DeepSeek-V4-Pro and Qwen3.7-Plus, to approximate how models reason about permission usage during tool calls. APIs that fall outside the models' expectations indicate mismatches between a tool's visible description and its implementation-level behavior.

IV. Empirical Study

Based on the dataset and analysis pipeline described above, this section presents the empirical findings of our study. The study is organized around four research questions, each corresponding to one layer of the MCP permission problem.

RQ1. Where are MCP servers deployed, and how complex are their tool interfaces? This question establishes the ecosystem context for our study. We characterize MCP server scenarios and measure implementation size, exposed tools, and permission-relevant API usage to assess where permission risks may have a broader impact.

RQ2. What permission-relevant APIs do MCP tools exercise? This RQ examines what APIs MCP tools actually invoke during execution. Since local invocation context provides only limited clues, we retrieve online API documentation to recover each invocation's permission semantics, classify the underlying behaviors, and identify high-impact usages that can amplify agent failures or attacks.

RQ3. How are tool permissions authorized in practice? This RQ asks whether sensitive tool actions are protected by proper authorizations. We examine common mechanisms, including static credentials, token-based access, and OAuth-style authorization, and assess whether they are fine-grained enough to support least-privilege tool access.

RQ4. Do tool descriptions communicate permission-relevant behavior? This RQ evaluates whether agent-visible metadata matches the permissions exercised in server-side code. A mismatch can hide risky behavior from the agent and lead to unsafe tool selection, so we compare tool descriptions with implementation-derived permission behavior and test whether LLMs can infer the corresponding permission scopes.

A. Deployment Scenarios and Tool Complexity

RQ1 provides the ecosystem baseline for our permission analysis. We classify MCP servers by deployment scenario and measure their complexity along three dimensions, including implementation size, exposed tool count, and permission-relevant API usage. These results show where MCP servers are used and where permission risks may have a broader impact.

Deployment Scenarios. This section first characterizes the deployment scenarios of the collected MCP servers. We identify eight scenario categories through an independent sampling and merging process, as described in Section III-C. For each of

the 4,049 collected servers, we extract project context from README files, metadata files, and a shallow directory tree, and use an LLM to generate a structured summary, capability descriptors, and type labels. The results are shown in Table I.

TABLE I: Distribution of MCP server categories.

Type Name	Description	Count
Data and Public Content Access	Public content, databases, documents, and knowledge bases	1607
Infrastructure and Gateway Services	Cloud, containers, gateways, and deployment operations	692
Source Control and Collaboration	Repositories, pull requests, and code review workflows	589
Task and Schedule Management	Tasks, tickets, calendars, and scheduling workflows	431
Local Execution and Automation	Local commands, file systems, and workspace automation	375
Financial and Blockchain Services	Payments, trading, market data, and blockchain services	143
Analytics and Peripheral Tools	Computation, visualization, media, and device tools	130
Security, Access Control, and Scanning	Security auditing, permissions, secrets, and vulnerability scanning	82
Total		**4049**

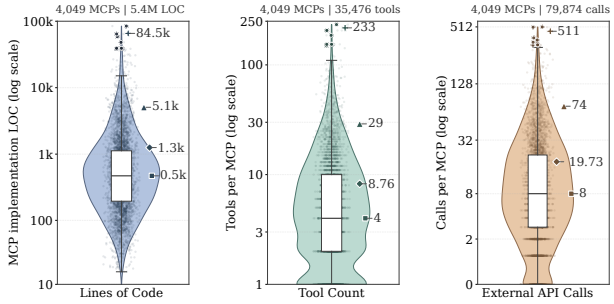
Among all categories, Data and Public Content Access is the largest one, covering 1,607 servers (39.7%). This result shows that MCP is frequently used to connect agents with external data sources, not only to execute actions on external systems. Although read-oriented tools are less privileged than state-changing tools, they can still be security-sensitive when they expose private data or introduce untrusted content into the agent's context.

The remaining categories are more critical. The second largest category is Infrastructure and Gateway Services, with 692 servers (17.1%). These servers often support cloud infrastructure management or gateway management, where tool actions can change running services, routing rules, or infrastructure configuration. Such permissions are especially risky because a mistaken or manipulated invocation may disrupt availability, expose internal services, or alter production environments. This highlights the need for stricter control over MCP tool call and permission behavior.

Beyond these two dominant categories, MCP servers are also widely used in Source Control and Collaboration, Task and Schedule Management, and Local Execution and Automation. This spread shows that MCP is reaching a broad range of everyday computing workflows, from software development to local automation and service coordination. In these settings, poorly scoped permissions may lead to code changes, unsafe system actions, or exposure of personal data.

Finding 1: MCP servers are already deployed in security-sensitive scenarios, such as infrastructure management, local command execution, and financial services, which together account for 60.3% of the studied servers.

Complexity Analysis. We examine MCP server complexity along three dimensions, including code size, the number of exposed tools, and the number of permission-relevant APIs invoked by each server. These metrics capture both implementation complexity and functional breadth. More complex MCP servers may integrate more permission-bearing functionality, increasing their security risk and potential impact. Results are shown in Figure 3.



(a) Lines of Code. (b) Tools. (c) API Calls.

Fig. 3: Complexity Analysis from Three Aspects. ■: median; ♦: mean; ▲: 95th percentile; +: maximum.

Code Size Analysis. Figure 3a shows the distribution of lines of code (LOC) across the analyzed MCP servers. On average, a server contains 1,333 LOC. Among them, 16 servers exceed 30k LOC, indicating that the ecosystem also contains a noticeable number of much larger systems. This indicates that many MCP servers have grown beyond lightweight tool wrappers. Moreover, at the upper end, `adr-analysis-server`, a suite of MCP servers for AWS development and infrastructure workflows, contains 84,500 LOC. These larger servers often combine many functions and permission-relevant operations in one codebase, while spreading permission logic across modules and execution paths, making tool authority harder to audit and constrain.

At the other end, 51.6% of MCP servers contain fewer than 500 LOC, showing that many implementations remain small and often act as thin wrappers over underlying APIs. However, compact implementations do not necessarily provide fine-grained authorization. Instead, their limited codebase may omit the additional logic needed for detailed permission configuration and enforcement.

MCP Tool Quantity Analysis. The pipeline identifies 35,476 MCP tools from 4,049 MCP servers. Figure 3b shows the distribution of the number of tools exposed by each MCP server on a log-scaled axis. Overall, most servers expose a modest number of tools. 59.2% of servers expose between two and ten tools, suggesting that many MCP servers are compact and functionally focused rather than concentrating many capabilities in a single server. However, the distribution is strongly right-skewed. The mean number of tools reaches 8.76, more than twice the median, indicating that a small number of high-tool-count servers substantially increases the average. Furthermore, 929 MCP servers integrate more than

TABLE II: Categories of permission-relevant APIs and representative examples

Category	Count	Description	Examples	Share
Specialized Domain Data Services	8309	Access domain-specific external data.	*GoogleMaps.geocode()*, *YouTube.videos.list()*	30.3%
Project Management Services	3736	Manage issues, tasks, and repositories.	*JIRA.create_issue()*, *GitHub.createPullRequest()*	13.6%
Database Management	3209	Query, modify, or administer databases.	*SQLite.execute()*, *PostgreSQL.query()*	11.7%
Cloud Infrastructure Management	3035	Provision or manage cloud resources.	*EC2.describe_volumes()*, *Kubernetes.list_pod()*	11.1%
System Command Execution	2202	Run local commands or scripts.	*subprocess.run()*, *child_process.spawn()*	8.1%
Financial and Blockchain Services	1899	Process financial or blockchain transactions.	*Stripe.paymentIntents.create()*, *viem.sendTransaction()*	6.9%
Identity and Access Management	1695	Authenticate users and manage access.	*OAuth.run_server()*, *Credentials.refresh()*	6.2%
File System Operations	1563	Read, write, or transfer files.	*os.remove()*, *fs.writeFile()*	5.7%
AI and Machine Learning Services	1000	Invoke AI/ML models or services.	*OpenAI.chat.create()*, *OpenAI.embeddings()*	3.6%
Browser Automation and Web Interaction	817	Automate browsers and web pages.	*Puppeteer.waitForSelector()*, *Playwright.goto()*	3.0%

ten tools, and even 16 MCP servers have more than 100 tools in one server. These servers require more fine-grained permission management because they expose agents to a larger set of callable operations and increase the chance that multiple permissions interact to cause a substantial impact.

Permission Usage Analysis. Through static analysis, we further characterize permission complexity by counting the permission-relevant external API calls reachable from each exposed MCP tool. Figure 3c shows the distribution of permission-relevant API calls per server on a log-scaled axis. Overall, most MCP servers and tools tend to focus on specific functionality and make a relatively small number of permission-relevant API calls. On average, each MCP server makes 19.73 permission-relevant API calls across its tools, while each MCP tool involves 2.25 such calls on average. Our detailed counts further show that 838 servers fall in the 10–20 range. This pattern is consistent with the recommended practice of avoiding excessive concentration of permissions within a single server or tool.

However, some servers still remain highly complex. The 95th percentile reaches 74 permission-relevant API calls, 123 servers exceed 100 such calls, and the largest server, `pocketbase-mcp-server` contains 511 API calls. This concentration of permission-relevant operations within a single server enlarges the authority surface exposed to agents, especially when tools share the common credential. These API calls may correspond to different permission levels or resource types, yet they are integrated into one server, which conflicts with the least-privilege principle. This concentration creates two risks. First, unintended behavior in one tool may affect other tools in the same server and amplify downstream impact. Second, an attacker or misaligned agent may compose multiple capabilities within one MCP server, rather than depending on a specific combination of separately installed servers.

Finding 2: Most MCP servers are functionally focused, but a small subset concentrates many tools and permission-relevant API usages within a single server. In particular, 123 servers make more than 100 API calls. Such concentration weakens least-privilege design and increases the risk of excessive permission combinations.

B. Permission-Relevant API Analysis

This section addresses RQ2 by analyzing what permissions MCP tools exercise in practice and which operations are most security-sensitive, and further reveals a mismatch between existing API safety assumptions and agent-driven tool use. Specifically, as described in Section III-B, permission behavior is first recovered for each API with online documentation retrieval. This step also deduplicates call sites towards the same API, yielding 27,465 unique permission-relevant APIs. Then, we identify ten permission categories through a sampling-based category discovery and merging workflow, as detailed in Section III-C. Finally, each API is categorized by providing the LLM with its recovered behavior and invocation context. The result is shown in Table II.

Some categories are comparatively less critical, including Specialized Domain Data Services, AI and ML Services, and Browser Automation, which together account for 36.9% of all APIs. Specialized Domain Data Services is the largest category, with 8,309 APIs (30.3%), indicating that agents rely heavily on MCP tools to obtain domain-specific information from external sources. Although many of these APIs are read-oriented, they can still become sensitive when combined with other tool permissions, because retrieved data may include private information or be passed into later tool actions.

A large share of APIs support direct interaction with external systems. Project Management Services, Database Management, Cloud Infrastructure Management, System Command Execution, and File System Operations together account for 13,745 APIs, approximately 50.0%. These APIs can modify data, start or stop cloud resources, or execute commands. Their prevalence shows that developers are actively connecting MCP servers to operational scenarios, though their usage requires strong security awareness and strict permission control.

The remaining two categories account for 3,594 APIs, approximately 13.1%. Although smaller in share, they still include high-impact capabilities such as identity and access management, financial, and blockchain operations. Their lower prevalence may reflect more conservative automation of sensitive interfaces rather than lower risk.

Overall, a common pattern in MCP servers is to wrap existing service APIs as agent-callable tools. However, many of these APIs were designed for use in controlled applications or

deliberate automation workflows, where developers define the execution logic and users understand the operational context. Several real-world API documents directly state that callers are responsible for safe and appropriate use. Tesla's Fleet API [32] expects developers to retry vehicle commands cautiously and verify the resulting vehicle state, while Stripe [33] expects users to understand that invoice voiding is irreversible and may be constrained by local regulations. Twilio's Consent Management API [34] similarly leaves compliance judgment to integrators by excluding HIPAA-subject workflows. These examples indicate that the security models of many current API services implicitly rely on developers or integrators to judge whether each invocation is safe, but this assumption cannot be guaranteed when APIs are invoked by agents.

Finding 3: The MCP ecosystem has integrated diverse critical service APIs, such as database operations and infrastructure management, which account for 63.1% of all APIs. This poses a critical mismatch, where security-sensitive APIs originally designed for responsible users are now exposed to autonomous agents.

C. Authorization Analysis

This section addresses RQ3 by examining how authorization is implemented when MCP tools invoke external APIs. We characterize the distribution of authorization types and their adoption across MCP server categories, revealing recurring weaknesses in current authorization practices.

Authorization Types. We first characterize common authorization mechanisms for permission-relevant external-API calls, using a code agent to identify concrete implementation evidence. The pipeline identifies eight representative authorization mechanisms across 4,049 MCP servers. Figure 4 shows the distribution of these mechanisms.

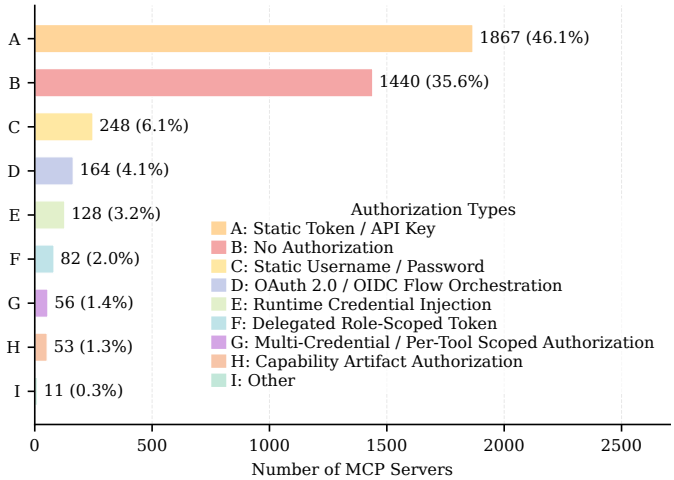


Fig. 4: Distribution of Authorization Mechanism.

No Authorization. Many MCP servers expose external operations without explicit egress authorization, especially in local deployments where the server is assumed to act on the user's behalf. In these cases, approving a tool call often grants the agent access to the server's ambient privileges without any finer-grained authorization check. This shifts permission enforcement from explicit access-control mechanisms to the LLM's ability to select safe actions.

Static Username and Password. More concerningly, 248 servers (6.1%) rely on plaintext username-password credentials when invoking external services, such as `mysql_mcp_server` and `applescript-mcp` [35], [36]. The former uses user-provided `MYSQL_USER` and `MYSQL_PASSWORD` credentials to run both read-only and state-changing SQL operations. The latter accepts SSH username-password credentials to execute AppleScript on a remote Mac host. Both examples show how one credential can confer broad authority over an external system. This pattern often grants the tool account-level authority over the corresponding service, rather than a scoped subset of permissions tied to a specific task. It therefore leaves little room for fine-grained restriction and makes credential exposure particularly damaging.

Static Token and API Key. This is the most common authorization type for MCP tools. Token-based authorization offers a convenient way to connect MCP tools to existing services, as many services already expose token-based interfaces. Developers can therefore integrate these services without modifying the service side.

However, such credentials are typically configured manually and are rarely scoped to the behavior of an individual tool. A single token often grants access to a broad API surface, even when the tool only requires a narrow subset of operations. Although some platforms provide partial scope separation, such as GitHub repository scopes, users may still choose broader or long-lived tokens to reduce configuration friction and avoid execution failures. As a result, token-based authorization can leave MCP servers with persistent authority that exceeds the needs of their individual tools.

For example, `Cognee` [37] initializes a shared Cognee client with a single service token. Then, this client is reused across tools with very different effects, including read-only tools such as `list_datasets` and `list_data`, and a destructive tool that invokes the "`DELETE /data`" endpoint [38]. Likewise, `square-mcp-server` [39] loads one `ACCESS_TOKEN` into a module-level Square client that serves tools for payments, invoices, and business operations. These examples show that token authorization is often bound to a shared client or server process, not to the risk profile of each tool. As a result, low-risk queries and high-impact mutations inherit the same broad authority, leaving the MCP server with coarse, persistent privileges that exceed least-privilege needs.

OAuth and OIDC Flow Orchestration. OAuth/OIDC provides a structured form of delegated authorization for accessing protected services. Instead of relying on a static credential, an MCP server obtains an access token through an authorization flow and uses it when a tool invokes the external APIs. Because such tokens can be scoped to specific operations and issued with limited lifetimes, this pattern is generally regarded as a best practice for delegated access

and aligns more closely with the principle of least privilege than static credentials. Among 4,049 servers, it appears in only 164 servers (4.1%), largely due to deployment cost and service-side dependencies. Representative examples include `google_workspace_mcp` [40], which supports Google OAuth flows and token refresh for Workspace access, and `mcp-atlassian` [41], which uses Atlassian Cloud authorization flow. These examples illustrate that OAuth/OIDC-based MCP servers can delegate authorization to the underlying service provider and avoid embedding long-lived static credentials directly in server configuration.

However, such deployments remain relatively uncommon in the current MCP ecosystem. A complete OAuth/OIDC setup requires authorization endpoints, token issuance, and scope-aware resource enforcement, which MCP developers can adopt only when the underlying service supports delegated authorization. Both examples above rely on vendors' mature OAuth infrastructure and service-side authorization support. As a result, OAuth/OIDC offers a clearer path toward fine-grained authorization but remains far less common than static credentials in current MCP deployments.

Other Fine-Grained Mechanisms. A similar deployment gap appears in Delegated Role-Scoped Token, Multi-Credential / Per-Tool Scoped Authorization, and Capability Artifact Authorization, which together remain rare in the ecosystem. These mechanisms better support the least privilege by binding authority to roles, scopes, capabilities, or tool-specific boundaries, rather than placing a single broad credential inside the server. Yet they also demand more deliberate authorization design, stronger platform support, or additional implementation effort. Their limited adoption suggests that fine-grained authorization remains difficult to deploy in practice.

Finding 4: Authorization in current MCP servers is largely absent or coarse-grained, with 3,555 servers (87.8%) relying on static tokens, passwords, or no authorization. This is due to a systemic gap where many API service backends do not support fine-grained delegated authorization.

Authorization Mechanisms Across Server Types. We further examine how authorization mechanisms are distributed across MCP server categories. This analysis helps reveal whether existing authorization practices are aligned with the collaboration patterns and risk profiles of different server types.

As shown in Figure 5, the two dominant patterns, API-key-based authorization and no explicit authorization, appear across nearly all major categories rather than being confined to low-risk deployments. This broad spread includes critical categories, where static-token authorization appears in 48.0% of infrastructure management servers and 30.4% of local execution servers. This distribution reveals a key mismatch in current MCP practice that authorization mechanisms do not become more precise as tool authority becomes more consequential, leaving many sensitive operations governed by static credentials or coarse user approval.

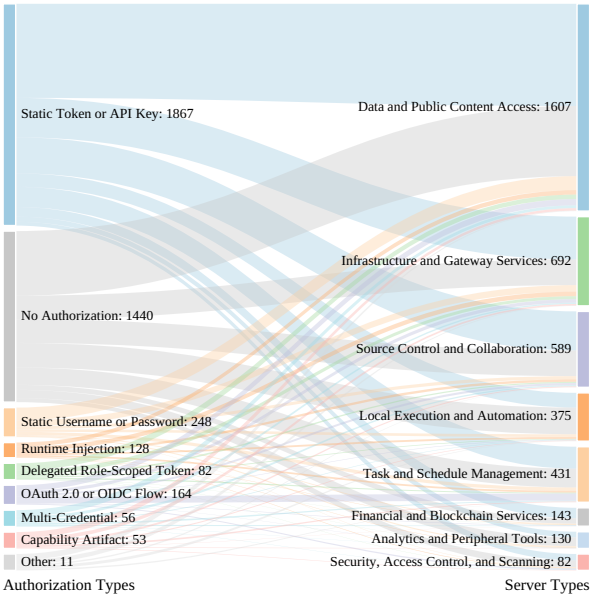


Fig. 5: Distribution of Authorization Types Across MCP Server Categories.

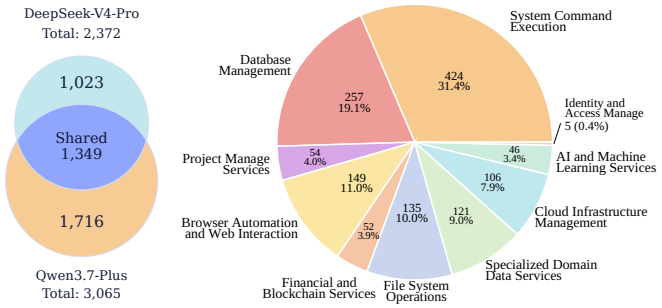
Fine-grained mechanisms, such as OAuth/OIDC and delegated tokens, remain marginal in Figure 5. Together, they account for only 246 cases, and their flows are thin even in high-consequence categories such as Infrastructure and Gateway Services (50 cases). This suggests that current MCP deployments rarely add tool-level authorization on top of underlying APIs. Instead, sensitive tools are still often governed by ambient credentials or coarse approval.

Finding 5: MCP authorization does not scale with risk. Static credentials remain common in high-consequence deployments, including infrastructure management (48.0%) and local command execution (30.4%).

D. Description–Permission Consistency Analysis

This section examines whether MCP tool descriptions faithfully expose the permission behaviors exercised by their underlying implementations. This question is important because, during MCP tool selection, the LLM typically observes only the tool name, parameter schema, and natural-language description. The concrete execution behavior, including API calls that access the file system or operate on external services, remains hidden behind the server-side tool entry point. If the description diverges from the implemented permission behavior, the agent may fail to recognize the risks associated with a tool call, leading to unsafe tool selection and potentially harmful actions.

We analyze all 4,049 servers, covering 35,476 MCP tools. For each tool, we compare the LLM-visible description with the permission-relevant APIs identified from their implementation. We then ask two LLMs, DeepSeek-V4-Pro and Qwen3.7-Plus, whether the exercised permission categories can be inferred from the description alone.



(a) (b)

Fig. 6: Shared blind spots in permission-relevant APIs, showing the overlap between two models in (a) and the category distribution of shared APIs in (b)

Cross-Model Consistency Analysis. Overall, we identify 2,219 tools for which both models consistently find that the descriptions do not accurately cover the permission behaviors exercised during execution, accounting for 6.3% of all analyzed tools. Specifically, DeepSeek-V4-Pro identifies 3819 tools with unexpected permission behaviors, while Qwen3.7-Plus identifies 4168 such tools. At the permission-record level, as shown in Figure 6a, 2,372 and 3,065 permission-relevant API invocations are not reflected in tool descriptions according to DeepSeek-V4-Pro and Qwen3.7-Plus, respectively. The large number of omitted permission-relevant API invocations suggests that tool descriptions are often incomplete or ambiguous, leaving room for different LLMs to interpret the same description differently. More importantly, 1,349 permission-relevant API invocations are omitted by both models. These shared omissions form critical API-level permission blind spots, because they are likely to remain invisible to LLM agents during tool selection. Together, these results indicate a critical description–a permission gap, where many tools describe their intended functionality but fail to disclose the permissions they exercise.

Finding 6: 2,219 MCP tools (6.3%) have descriptions that omit permissions exercised during execution, creating systematic blind spots that may cause LLM agents to underestimate tool risks during selection.

Category Distribution of Omitted APIs. When examining the category distribution of these shared omitted permission-relevant APIs, as shown in Figure 6b, we find that its distribution largely shifts toward high-impact categories compared to the overall distribution in Table II. Specifically, system command execution accounts for the largest share of shared omitted APIs, reaching 31.4%, even though it represents only 8.0% of all permission-relevant APIs. Database management is the second largest omitted category, despite ranking third in the overall distribution. These categories involve high-impact operations whose misuse can lead to system compromise, data loss, or service disruption. This is because

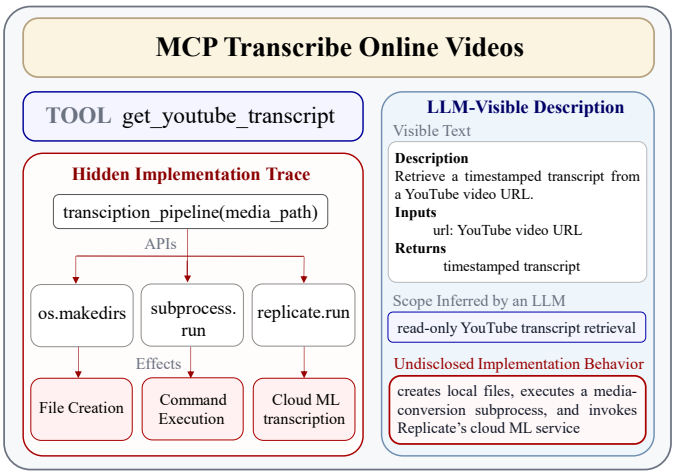


Fig. 7: An example of description-behavior mismatch. The tool is described as read-only YouTube transcript retrieval, but its implementation exercises hidden permissions.

the description–permission gap is more pronounced for high-impact operations, whose risks become easier for LLMs to recognize once their underlying behaviors are made explicit. Overall, omitted permission-relevant APIs are concentrated in high-impact categories, suggesting that MCP tool descriptions often under-disclose consequential permission behaviors.

Figure 7 illustrates a representative description-behavior mismatch. The tool `get_youtube_transcript`, exposed by the `mcp_transcribe_online_vids` server, is described as taking a YouTube URL and returning a timestamped transcript. Based on this metadata, an LLM would likely infer a narrow read-only capability for retrieving transcript text, without expecting local state changes or high-impact service invocation. However, the hidden execution trace calls `transcription_pipeline(media_path)`, which further invokes `os.makedirs`, `subprocess.run`, and `replicate.run`. These APIs correspond to file-system operations, system command execution, and cloud service invocation. The mismatch arises because the description states the tool's functional goal but omits the operational steps needed to realize it, such as creating local directories, running commands, and invoking Replicate for transcription. As a result, the model may treat the tool as a low-risk transcript reader, while the implementation exercises broader and more security-sensitive permissions.

Finding 7: Many omitted APIs are high-impact, with 31.4% involving command execution and 19.1% involving database management. This suggests that the description–permission gap usually hides security-critical permissions.

V. Implications of Study

Toward Fine-Grained Permission Separation. Our findings suggest that external services need to evolve from user-oriented authorization toward agent-oriented authorization. Existing models are often designed for human readability and administrative convenience, grouping authority into coarse

categories such as read and write access. For example, although Linear adopts an OAuth workflow, it exposes only a small set of permission categories, such as read, write, and `comments:create` [42]. Such coarse grouping is manageable for human users but poorly matched to dynamic agent execution, where a model may select, combine, and invoke tools across changing task contexts. Therefore, agent-facing services should expose finer-grained scopes over specific entities and actions, allowing authorization boundaries to better match the capabilities that agents actually exercise during tool selection and execution.

Toward Fine-Grained Permission Control. Many MCP servers still rely on static credentials, causing a server to hold broad authority for long periods even when individual tools need only narrow access. A better direction is for service providers to gradually support OAuth-style delegated authorization, short-lived tokens, and fine-grained scopes aligned with concrete tool behavior. Such migration would reduce over-privilege and make permission management more suitable for agent-driven workflows. However, it requires substantial adaptation effort from service providers, SDK maintainers, and MCP server developers, which may slow broader adoption. This also calls for new authorization paradigms that can bridge traditional static-token management in extensive existing services with the highly dynamic permission requirements of agent-driven execution.

Permission Disclosure Tradeoffs. Our results show that tool descriptions often fail to reflect the API-level behavior executed by the underlying implementation. A direct response is to add permission-relevant information so agents can better assess a tool before invocation. However, adding many tools and detailed descriptions can lengthen the input context and degrade model decision quality. This creates a tradeoff between disclosure granularity and reasoning effectiveness. Addressing it requires calibrating descriptions to expose security-relevant behavior at the right granularity, rather than simply adding detail. MCP systems may also adopt progressive disclosure, showing concise metadata by default and providing finer-grained permission semantics only when needed or when a tool carries elevated risk.

VI. Threats to Validity

First, this paper focuses on MCP servers developed in Python and TypeScript, which dominate the current public MCP ecosystem but do not cover all implementations. MCP servers also appear in other languages; for instance, Java is often used for integration with enterprise back-end systems, while Go is common in cloud-facing or infrastructure-oriented services. Based on our unified workflow for tool extraction and call-graph construction, the analysis can be extended to these languages. We plan to support more languages in future work.

Second, several stages of our study rely on LLM-assisted semantic analysis, such as permission-relevant API identification and category classification. This introduces a risk of misclassification, especially for uncommon APIs. We mitigate this threat through three safeguards. First, we augment permission analysis with online documentation retrieval, providing the model with API-specific context. Second, we use repeated sampling when constructing categories to reduce instability in model-generated labels. Third, we manually inspect sampled results and uncertain cases to check whether the inferred semantics are grounded in the available evidence. Together, these mitigations limit the effect of remaining errors, so that they will not materially affect the overall conclusions.

VII. Related work

Prior work has explored security and safety issues in agent ecosystems from different perspectives. First, prior work establishes LLM-based agents as a distinct class of software systems. Thus, many studies have investigated their design, implementation, and execution. Some surveys summarize their core design dimensions, such as planning, memory, tool use, and environmental interaction [6], [7]. System-oriented studies show how agents can be implemented and evaluated in practical settings, from multi-agent frameworks to software engineering and web-based tasks [8], [43], [44].

Some work focuses more directly on protocols for agent interoperability. Ehtesham et al. survey emerging protocols, including MCP, ACP, A2A, and ANP, and compare how they support agent-tool and agent-agent interaction [13], [45]–[48]. Hou et al. study the MCP ecosystem from the perspectives of architecture, security threats, and future research directions [14]. Unlike these studies, our work focuses specifically on permission issues in MCP server implementations.

A growing body of work studies the security and safety risks of LLM agents. Recent surveys summarize risks introduced by autonomy and tool use, such as data leakage, privacy violations, and unsafe external actions [9]–[12]. They categorize these risks by system component and execution stage, showing where agent failures may occur. Another line of work builds benchmarks and evaluations for security and safety in tool-integrated agents [15], [16], [49], [50]. These benchmarks use task scenarios, attack prompts, and evaluation suites to measure whether agents resist unsafe instructions or avoid harmful tool use. Overall, unlike prior work on concrete attacks, defenses, or agent-level safety behavior, our work examines permission issues in MCP server implementations.

VIII. Conclusion

This paper presented a large-scale empirical study of permission control in MCP-based LLM agent tools. Our analysis first identifies the concrete permission-relevant API invocations made by each MCP tool, and then uses these calls as evidence for analyzing permission usage and authorization practices. By analyzing 4,049 open-source MCP servers and 35,476 tools, we identify several key findings. First, immature MCP server practices obscure and concentrate high-impact permission behavior. Second, human-oriented API services often expose coarse or absent authorization when integrated with automated agents. Together, these problems cause a security governance deficit in current MCP deployments.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[2] M. Li, F. Song, B. Yu, H. Yu, Z. Li, F. Huang, and Y. Li, "API-bank: A comprehensive benchmark for tool-augmented LLMs," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 3102–3116. [Online]. Available: https://aclanthology.org/2023.emnlp-main.187

[3] Anthropic, "Model Context Protocol specification," https://modelcontextprotocol.io/specification/, 2024.

[4] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024.

[5] Y. Ruan, C. J. Maddison, T. Hashimoto, and P. Liang, "ToolEmu: Identifying the risks of LM agents with an LM-emulated sandbox," in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=ryjYQe8YwH

[6] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[7] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.

[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversations," in *First conference on language modeling*, 2024.

[9] H. Su, J. Luo, C. Liu, X. Yang, Y. Zhang, Y. Dong, and J. Zhu, "A survey on autonomy-induced security risks in large model-based agents," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026, early Access.

[10] X. Ma, Y. Gao, Y. Wang, R. Wang, X. Wang, Y. Sun, Y. Ding, H. Xu, Y. Chen, Y. Zhao *et al.*, "Safety at scale: A comprehensive survey of large model and agent safety," *Foundations and Trends in Privacy and Security*, vol. 8, no. 3–4, pp. 1–240, 2025.

[11] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The emerged security and privacy of LLM agent: A survey with case studies," *ACM Computing Surveys*, 2025.

[12] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "AI agents under threat: A survey of key security challenges and future pathways," *ACM Computing Surveys*, 2025.

[13] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp)," *arXiv preprint arXiv:2505.02279*, 2025.

[14] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (mcp): Landscape, security threats, and future research directions," *ACM Transactions on Software Engineering and Methodology*, 2025.

[15] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 10 471–10 506. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[16] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[17] MCP.so, "MCP Servers," https://mcp.so/, accessed: 2026-07-01.

[18] Cursor Directory, "Cursor Directory: Plugins for cursor," https://cursor.directory/, accessed: 2026-07-01.

[19] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.

[20] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 744–20 757, 2022.

[21] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," *Advances in Neural Information Processing Systems*, vol. 36, pp. 28 091–28 114, 2023.

[22] F. Schneider, "Least privilege and more [computer security]," *IEEE Security & Privacy*, vol. 1, no. 5, pp. 55–59, 2003.

[23] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[24] P. Wijesekera, A. Baokar, A. Hosseini, S. Egelman, D. Wagner, and K. Beznosov, "Android permissions remystified: A field study on contextual integrity," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 499–514.

[25] D. Hardt, "The OAuth 2.0 authorization framework," RFC 6749, 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749

[26] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of oauth 2.0," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 1204–1215.

[27] K. Luo, X. Wang, P. H. A. Fung, W. C. Lau, and J. Lecomte, "Universal cross-app attacks: Exploiting and securing {OAuth} 2.0 in integration platforms," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3221–3238.

[28] M. B. Jones, P. Hunt, and A. Parecki, "Oauth 2.0 protected resource metadata," Internet Engineering Task Force, RFC 9728, April 2025. [Online]. Available: https://www.rfc-editor.org/info/rfc9728

[29] D. Fett, P. Hosseyni, and R. Küsters, "An extensive formal security analysis of the openid financial-grade api," *arXiv preprint arXiv:1901.11520*, 2019.

[30] D. Klopp, S. Erdweg, and A. Pacak, "Object-oriented fixpoint programming with datalog," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024. [Online]. Available: https://doi.org/10.1145/3689713

[31] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented Queries on Relational Data," in *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), S. Krishnamurthi and B. S. Lerner, Eds., vol. 56. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, pp. 2:1–2:25. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2016.2

[32] Tesla, "Fleet api best practices," https://developer.tesla.com/docs/fleet-api/getting-started/best-practices, 2026.

[33] Stripe, "Stripe api-void an invoice," https://docs.stripe.com/api/invoices/void, 2026.

[34] Twilio, "Twilio consent management api," https://www.twilio.com/docs/messaging/features/consent-api, 2026.

[35] designcomputer, "MySQL MCP Server," https://github.com/designcomputer/mysql_mcp_server, 2026.

[36] peakmojo, "AppleScript MCP Server," https://github.com/peakmojo/applescript-mcp, 2026.

[37] topoteretes, "Cognee MCP Server Implementation," https://github.com/topoteretes/cognee/blob/main/cognee-mcp/src/server.py, 2026.

[38] ——, "Cognee MCP Client Implementation," https://github.com/topoteretes/cognee/blob/main/cognee-mcp/src/cognee_client.py, 2026, accessed: 2026-06-25.

[39] square, "Square Model Context Protocol Server," https://github.com/square/square-mcp-server, 2026.

[40] taylorwilsdon, "Google Workspace MCP Server," https://github.com/taylorwilsdon/google_workspace_mcp, 2026.

[41] sooperset, "MCP Atlassian," https://github.com/sooperset/mcp-atlassian, 2026.

[42] Linear, "OAuth 2.0 authentication-Linear Developers," https://linear.app/developers/oauth-2-0-authentication, 2026.

[43] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, S. Yau, Z. Lin, L. Zhou *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 23 247–23 275.

[44] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "Webarena:

A realistic web environment for building autonomous agents," 2024. [Online]. Available: https://arxiv.org/abs/2307.13854

[45] A. Sarker and X. Wang, "From code understanding to multi-agent collaboration: A layered protocol stack for modern ai intelligent systems," in *2026 IEEE World AI IoT Congress (AIIoT)*. IEEE, 2026, pp. 0910–0916.

[46] G. Chang, E. Lin, C. Yuan, R. Cai, B. Chen, X. Xie, and Y. Zhang, "Agent network protocol technical white paper," 2025. [Online]. Available: https://arxiv.org/abs/2508.00007

[47] N. K. Krishnan, "Beyond context sharing: A unified agent communication protocol (acp) for secure, federated, and autonomous agent-to-agent (a2a) orchestration," 2026. [Online]. Available: https://arxiv.org/abs/2602.15055

[48] A2A Protocol Working Group, "Agent2Agent (A2A) Protocol Specification," https://a2a-protocol.org/latest/specification/, 2026, accessed: 2026-07-01.

[49] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 35 331–35 366.

[50] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson *et al.*, "Agentharm: A benchmark for measuring harmfulness of llm agents," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 79 185–79 220.